

EXPERIMENT - 6

AIM:

To implement a Logistic Regression classifier, obtain predicted probabilities, study the effect of different classification thresholds on precision and recall, analyze confusion matrices, and use ROC and Precision-Recall curves to evaluate classification performance.

THEORY:

Logistic Regression

Logistic regression is a supervised learning algorithm used for binary classification. It first computes a linear score $z = b_0 + b_1x_1 + \dots + b_nx_n$ and then transforms that score into a probability using the sigmoid function. The model is linear in the log-odds, which makes its fitted coefficients directly interpretable.

$$p(y = 1 | x) = 1 / (1 + \exp(-z))$$

Predicted Probability and Classification Threshold

The sigmoid output lies between 0 and 1. A classification threshold converts this probability into a class label. At the default threshold of 0.50, probabilities at or above 0.50 are predicted as the positive class. Lower thresholds usually increase sensitivity but also increase false positives; higher thresholds usually increase specificity but may miss more positive cases.

Confusion Matrix and Classification Metrics

A confusion matrix records true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP). Accuracy measures the overall fraction of correct predictions. Precision measures how many predicted positives are correct. Sensitivity, also called recall, measures how many actual positives are detected. Specificity measures how many actual negatives are correctly rejected, while the F1-score balances precision and sensitivity.

Metric	Formula	Meaning
Accuracy	$(TP + TN) / \text{Total}$	Overall correctness
Precision	$TP / (TP + FP)$	Reliability of positive predictions
Sensitivity	$TP / (TP + FN)$	Ability to find positive cases
Specificity	$TN / (TN + FP)$	Ability to reject negative cases
F1-score	$2PR / (P + R)$	Balance of precision and recall

ROC and Precision-Recall Curves

The ROC curve plots the true-positive rate against the false-positive rate over all thresholds. ROC-AUC summarizes how well predicted scores rank positive cases above negative cases. The Precision-Recall curve plots precision against recall and is especially useful when attention is focused on the positive class. Average precision summarizes this curve and should be compared with the positive-class prevalence.

CODE:

```
# EXPERIMENT 6: Logistic Regression, Thresholds and Classification Metrics

# 2. Import libraries
from pathlib import Path
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay, accuracy_score, average_precision_score,
    classification_report, confusion_matrix, f1_score,
    precision_recall_curve, precision_score, recall_score,
    roc_auc_score, roc_curve,
)
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 42
sns.set_theme(style="whitegrid", context="notebook")
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", lambda value: f"{value:.4f}")

# 3. Load and audit the dataset
file_name = "heart disease classification dataset.csv"
candidate_paths = [Path(file_name), Path("Week 6") / file_name]
data_path = next((path for path in candidate_paths if path.exists()), None)
if data_path is None:
    raise FileNotFoundError(f"Could not find {file_name!r} in the notebook directory or Week 6/.")

raw_data = pd.read_csv(data_path)
index_columns = [column for column in raw_data.columns if column.startswith("Unnamed:")]
data = raw_data.drop(columns=index_columns).copy()
duplicate_rows = int(data.duplicated().sum())
if duplicate_rows:
    data = data.drop_duplicates().reset_index(drop=True)

target_map = {"no": 0, "yes": 1}
data["target"] = data["target"].str.strip().str.lower().map(target_map)

print(f"Loaded: {data_path.resolve()}")
print(f"Raw shape: {raw_data.shape}")
print(f"Removed saved-index columns: {index_columns or 'none'}")
print(f"Removed exact duplicate rows from the analysis copy: {duplicate_rows}")
print(f"Analysis shape: {data.shape}")
data.head()

quality_summary = pd.DataFrame({
    "dtype": data.dtypes.astype(str),
    "missing": data.isna().sum(),
    "unique": data.nunique(dropna=False),
})
display(quality_summary)

target_summary = (
    data["target"].value_counts().sort_index()
    .rename(index={0: "No disease", 1: "Disease"})
    .rename_axis("class").to_frame("rows")
)
target_summary["percent"] = 100 * target_summary["rows"] / len(data)
display(target_summary)
data.describe().T

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.countplot(data=data, x="target", hue="target", palette="Set2", legend=False, ax=axes[0])
axes[0].set(title="Target distribution", xlabel="Heart disease (0 = no, 1 = yes)", ylabel="Rows")

sns.histplot(data=data, x="age", hue="target", bins=15, element="step", stat="density",
             common_norm=False, ax=axes[1])
axes[1].set(title="Age distribution by class", xlabel="Age (years)")

sns.boxplot(data=data, x="target", y="thalach", hue="target", palette="Set2", legend=False,
            ax=axes[2])
axes[2].set(title="Maximum heart rate by class", xlabel="Heart disease", ylabel="Maximum heart rate (bpm)")

plt.tight_layout()
plt.show()

# 4. Define features and create a stratified split
continuous_features = ["age", "trestbps", "chol", "thalach", "oldpeak"]
categorical_features = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]
feature_columns = continuous_features + categorical_features

X = data[feature_columns].copy()
y = data["target"].astype(int).copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

split_summary = pd.DataFrame({
    "rows": [len(X_train), len(X_test)],
    "positive_rows": [int(y_train.sum()), int(y_test.sum())],
    "positive_rate": [y_train.mean(), y_test.mean()],
}, index=["Training", "Testing"])
display(split_summary)

# 5. Build leakage-safe preprocessing and models
numeric_pipeline = Pipeline([
```

CODE (CONTINUED):

```
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", drop="first")),
])
preprocessor = ColumnTransformer([
    ("continuous", numeric_pipeline, continuous_features),
    ("categorical", categorical_pipeline, categorical_features),
])
logistic_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)),
])
dummy_model = DummyClassifier(strategy="most_frequent")

dummy_model.fit(X_train, y_train)
logistic_model.fit(X_train, y_train)
print("Both models fitted successfully.")

# 6. Evaluate held-out performance
def evaluate_classifier(name, model, features, labels):
    predictions = model.predict(features)
    probabilities = model.predict_proba(features)[:, 1]
    tn, fp, fn, tp = confusion_matrix(labels, predictions, labels=[0, 1]).ravel()
    return {
        "Model": name,
        "Accuracy": accuracy_score(labels, predictions),
        "Precision": precision_score(labels, predictions, zero_division=0),
        "Sensitivity (Recall)": recall_score(labels, predictions, zero_division=0),
        "Specificity": tn / (tn + fp),
        "F1-score": f1_score(labels, predictions, zero_division=0),
        "ROC-AUC": roc_auc_score(labels, probabilities),
        "Average Precision": average_precision_score(labels, probabilities),
    }

models = {"Majority-class baseline": dummy_model, "Logistic regression": logistic_model}
test_results = pd.DataFrame([
    evaluate_classifier(name, model, X_test, y_test)
    for name, model in models.items()
]).set_index("Model")
test_results

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, (name, model) in zip(axes, models.items()):
    ConfusionMatrixDisplay.from_predictions(
        y_test, model.predict(X_test), labels=[0, 1],
        display_labels=["No disease", "Disease"], cmap="Blues", colorbar=False, ax=ax
    )
    ax.set_title(name)
plt.tight_layout()
plt.show()

logistic_predictions = logistic_model.predict(X_test)
print(classification_report(
    y_test, logistic_predictions, labels=[0, 1],
    target_names=["No disease", "Disease"], digits=4, zero_division=0
))

# 7. ROC and precision-recall curves
test_probabilities = logistic_model.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, test_probabilities)
precision_curve, recall_curve, _ = precision_recall_curve(y_test, test_probabilities)
roc_auc = roc_auc_score(y_test, test_probabilities)
average_precision = average_precision_score(y_test, test_probabilities)

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
axes[0].plot(fpr, tpr, linewidth=2, label=f"Logistic regression (AUC = {roc_auc:.3f})")
axes[0].plot([0, 1], [0, 1], "--", color="gray", label="Random ranking")
axes[0].set(title="ROC curve", xlabel="False-positive rate", ylabel="True-positive rate")
axes[0].legend()

axes[1].plot(recall_curve, precision_curve, linewidth=2, label=f"AP = {average_precision:.3f}")
axes[1].axhline(y=y_test.mean(), linestyle="--", color="gray", label=f"Prevalence = {y_test.mean():.3f}")
axes[1].set(title="Precision-recall curve", xlabel="Recall", ylabel="Precision")
axes[1].legend()

plt.tight_layout()
plt.show()

# 8. Five-fold cross-validation on the training set
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {
    "accuracy": "accuracy", "precision": "precision", "recall": "recall",
    "f1": "f1", "roc_auc": "roc_auc", "average_precision": "average_precision",
}
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", message="Found unknown categories.*", category=UserWarning)
cv_scores = cross_validate(logistic_model, X_train, y_train, cv=cv, scoring=scoring)
cv_summary = pd.DataFrame({
    metric.replace("test_", "").replace(" ", " ").title(): [values.mean(), values.std(ddof=1)]
    for metric, values in cv_scores.items() if metric.startswith("test_")
}, index=["Mean", "Standard deviation"])
cv_summary

# 9. Interpret logistic-regression coefficients
feature_names = logistic_model.named_steps["preprocessor"].get_feature_names_out()
coefficients = logistic_model.named_steps["classifier"].coef_[0]
coefficient_table = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients,
    "Odds ratio": np.exp(coefficients),
})
coefficient_table["Absolute coefficient"] = coefficient_table["Coefficient"].abs()
```

CODE (CONTINUED):

```
coefficient_table = coefficient_table.sort_values("Absolute coefficient",
ascending=False).reset_index(drop=True)
display(coefficient_table.head(12))

top_coefficients = coefficient_table.head(12).sort_values("Coefficient")
colors = np.where(top_coefficients["Coefficient"] >= 0, "#d95f02", "#1b9e77")
plt.figure(figsize=(9, 6))
plt.barh(top_coefficients["Feature"], top_coefficients["Coefficient"], color=colors)
plt.axvline(0, color="black", linewidth=1)
plt.title("Largest logistic-regression coefficients")
plt.xlabel("Coefficient (change in log-odds)")
plt.ylabel("Transformed feature")
plt.tight_layout()
plt.show()

# 10. Probability-threshold trade-off
threshold_rows = []
for threshold in [0.30, 0.40, 0.50, 0.60, 0.70]:
    threshold_predictions = (test_probabilities >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, threshold_predictions, labels=[0, 1]).ravel()
    threshold_rows.append({
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_test, threshold_predictions),
        "Precision": precision_score(y_test, threshold_predictions, zero_division=0),
        "Sensitivity": recall_score(y_test, threshold_predictions, zero_division=0),
        "Specificity": tn / (tn + fp),
        "F1-score": f1_score(y_test, threshold_predictions, zero_division=0),
        "Predicted positive rate": threshold_predictions.mean(),
    })
threshold_results = pd.DataFrame(threshold_rows).set_index("Threshold")
display(threshold_results)

threshold_results[["Sensitivity", "Specificity"]].plot(marker="o", figsize=(8, 4))
plt.ylim(0, 1.05)
plt.title("Sensitivity-specificity trade-off")
plt.ylabel("Score")
plt.tight_layout()
plt.show()

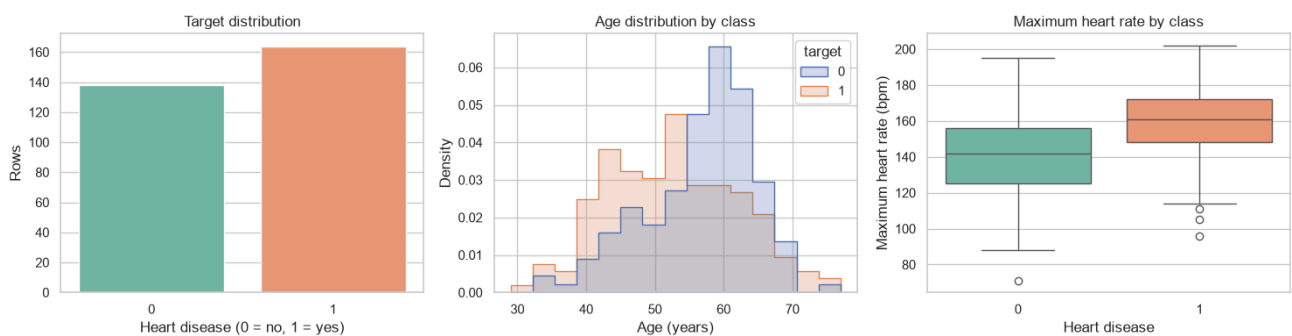
# 11. Automated checks
assert data.shape[1] == 14, "Unexpected number of analysis columns"
assert set(feature_columns) == set(data.columns) - {"target"}
assert y.isna().sum() == 0 and set(y.unique()) == {0, 1}
assert X_train.index.intersection(X_test.index).empty, "Train/test row overlap detected"
assert len(X_train) + len(X_test) == len(data)
assert abs(y_train.mean() - y_test.mean()) < 0.03, "Stratification check failed"
assert np.isfinite(test_probabilities).all()
assert ((test_probabilities >= 0) & (test_probabilities <= 1)).all()
assert test_results.loc["Logistic regression", "ROC-AUC"] > 0.5
assert test_results.loc["Logistic regression", "Accuracy"] > test_results.loc["Majority-class
baseline", "Accuracy"]
assert coefficient_table.shape[0] == len(feature_names)
print("All data, split, probability, performance, and coefficient checks passed.")
```

OUTPUT AND RESULTS:

1. Dataset Audit and Exploratory Analysis

The supplied CSV has 303 rows and 15 columns. The saved index column and one exact duplicate are removed only from the analysis copy, leaving 302 rows and 14 variables. Missing measurements occur in trestbps (4 rows), chol (1 row), and thalach (5 rows). The target contains 164 disease cases (54.30%) and 138 no-disease cases (45.70%).

Split	Rows	Positive rows	Positive rate
Training	241	131	0.5436
Testing	61	33	0.5410



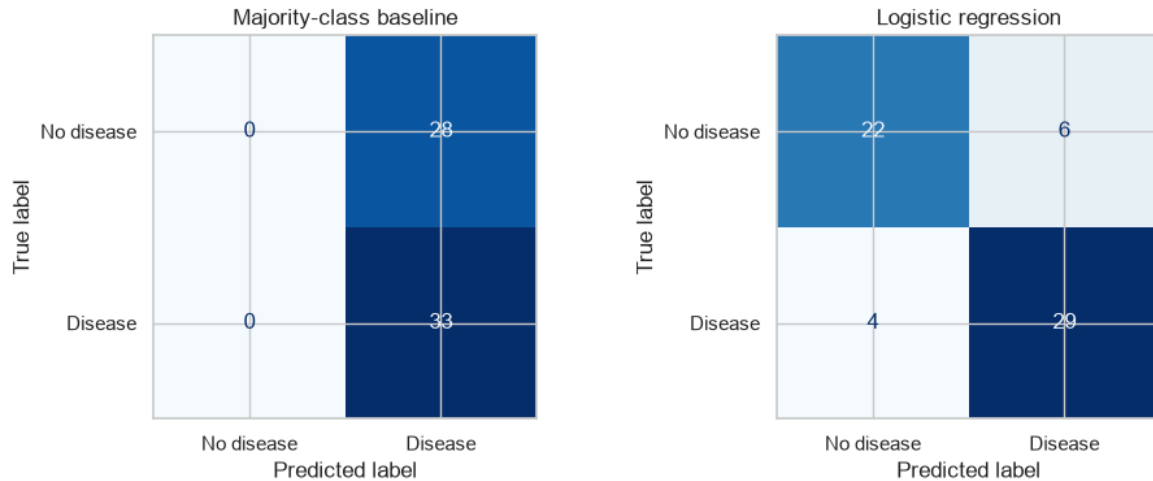
Target distribution, age distribution by class, and maximum heart rate by class.

2. Baseline Comparison and Held-Out Performance

Model	Acc.	Prec.	Sens.	Spec.	F1	ROC-AUC	AP
Majority baseline	0.5410	0.5410	1.0000	0.0000	0.7021	0.5000	0.5410
Logistic regression	0.8361	0.8286	0.8788	0.7857	0.8529	0.8853	0.9023

The majority-class baseline labels every test row as positive, so its 54.10% accuracy merely matches positive-class prevalence and its specificity is zero. Logistic regression improves accuracy by 29.51 percentage points and performs better on every informative metric.

3. Confusion Matrix and Per-Class Results

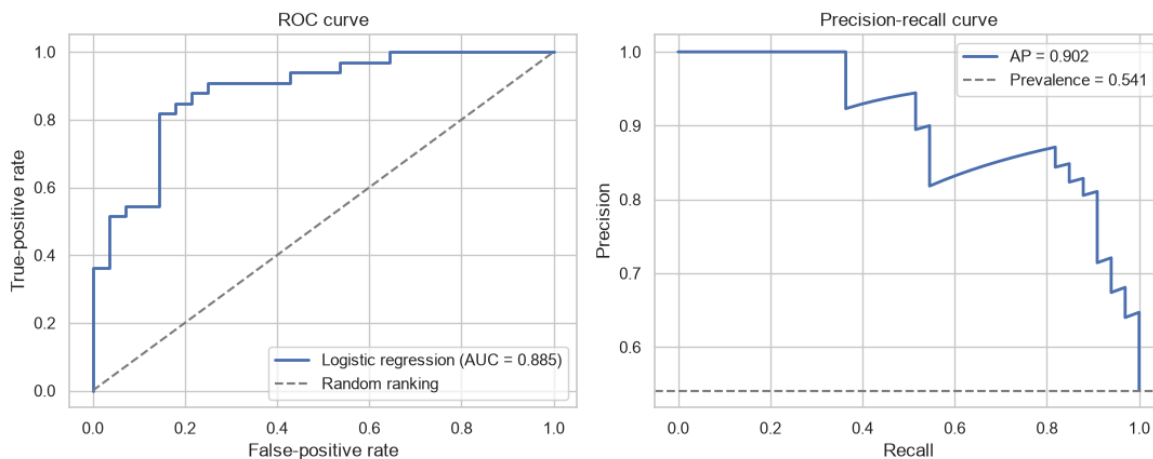


Confusion matrices for the majority-class baseline and logistic regression.

Actual class	Predicted no disease	Predicted disease
No disease	22	6
Disease	4	29

At the default threshold of 0.50, the model correctly classifies 51 of 61 test rows. It produces 6 false positives and 4 false negatives. The disease class has precision 0.8286, recall 0.8788, and F1-score 0.8529; the no-disease class has precision 0.8462, recall 0.7857, and F1-score 0.8148.

4. ROC and Precision-Recall Analysis



ROC curve and Precision-Recall curve for the logistic regression model.

The ROC-AUC of 0.8853 indicates useful ranking ability across thresholds. Average precision is 0.9023, substantially above the positive-class prevalence of 0.5410.

5. Five-Fold Cross-Validation

Cross-validation is performed only on the 241-row training portion. Every validation fold receives imputation, encoding, and scaling parameters learned from its corresponding training folds.

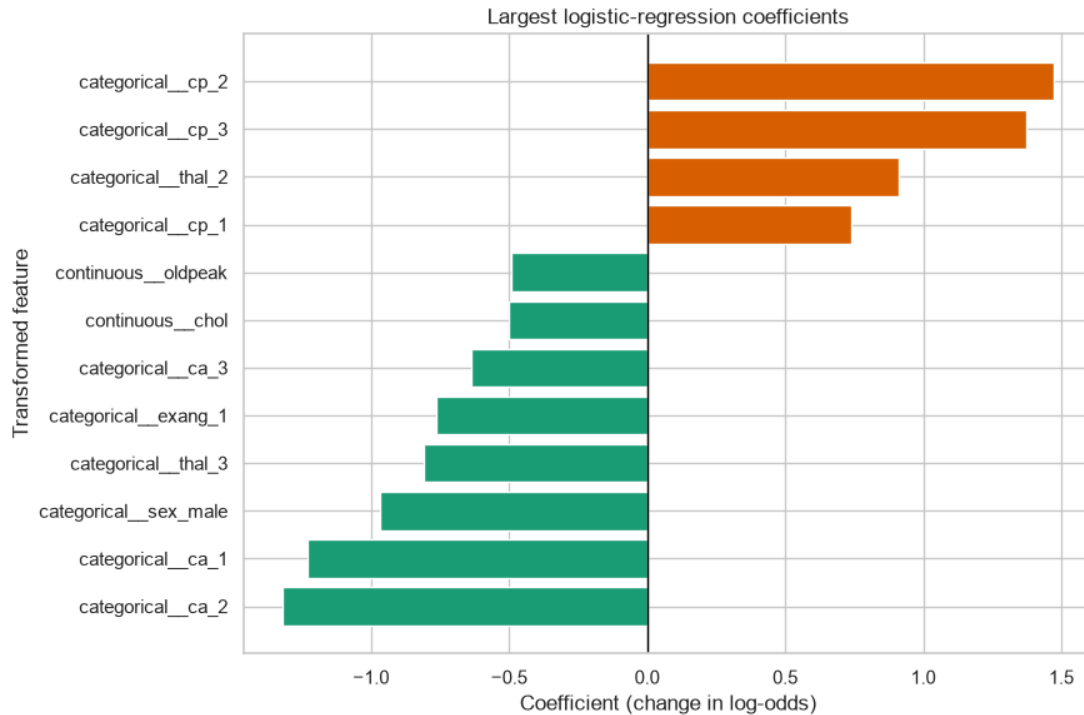
Metric	Mean	Standard deviation
Accuracy	0.8509	0.0568
Precision	0.8606	0.0623
Recall	0.8701	0.0586
F1-score	0.8641	0.0496
ROC-AUC	0.9088	0.0425
Average precision	0.9196	0.0295

The held-out results are consistent with these training-set estimates. The fold-to-fold variation is still meaningful because the dataset contains only 302 unique records.

6. Coefficient Interpretation

Transformed feature	Coefficient	Odds ratio
cp_2	1.4737	4.3653
cp_3	1.3751	3.9555
ca_2	-1.3196	0.2673
ca_1	-1.2280	0.2929
sex_male	-0.9670	0.3802
thal_2	0.9105	2.4856
thal_3	-0.8069	0.4462
exang_1	-0.7620	0.4667
cp_1	0.7374	2.0905
ca_3	-0.6383	0.5282
Standardized chol	-0.4996	0.6068
Standardized oldpeak	-0.4939	0.6103

6. Coefficient Interpretation (Continued)



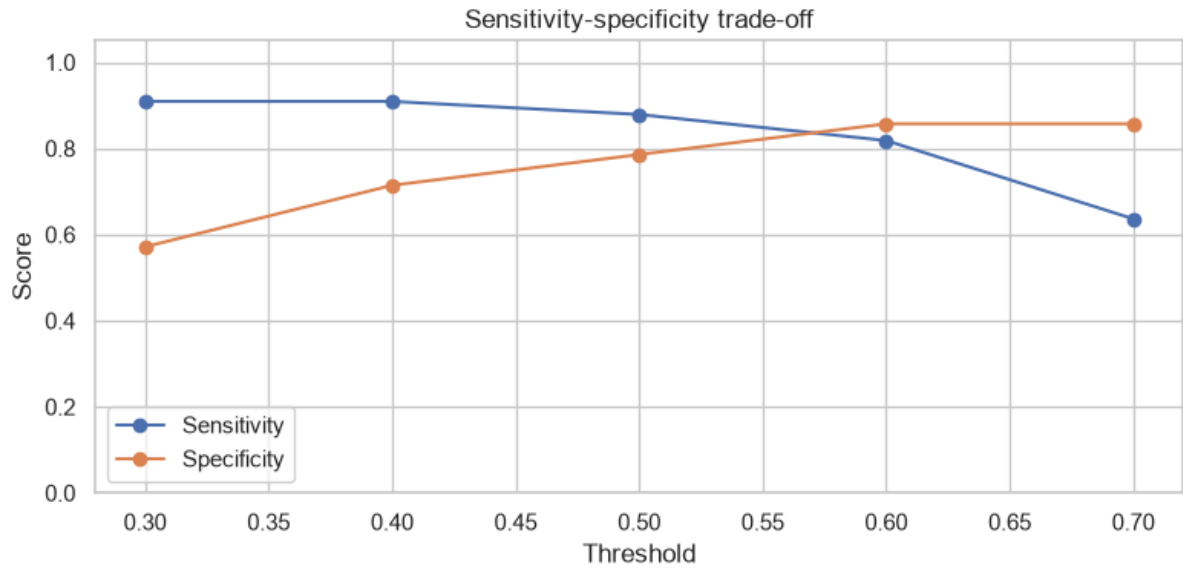
Largest fitted logistic-regression coefficients by absolute magnitude.

For a one-hot feature, the coefficient compares that category with the dropped reference category while other modeled variables stay fixed. For a standardized continuous feature, it corresponds to a one-standard-deviation increase. These values are fitted associations, not causal or clinical effects.

7. Probability-Threshold Analysis

Threshold	Accuracy	Precision	Sensitivity	Specificity	F1	Positive rate
0.30	0.7541	0.7143	0.9091	0.5714	0.8000	0.6885
0.40	0.8197	0.7895	0.9091	0.7143	0.8451	0.6230
0.50	0.8361	0.8286	0.8788	0.7857	0.8529	0.5738
0.60	0.8361	0.8710	0.8182	0.8571	0.8438	0.5082
0.70	0.7377	0.8400	0.6364	0.8571	0.7241	0.4098

7. Probability-Threshold Analysis (Continued)



Sensitivity-specificity trade-off as the probability threshold changes.

Lowering the threshold from 0.50 to 0.40 increases sensitivity from 0.8788 to 0.9091 but lowers specificity from 0.7857 to 0.7143. Raising it to 0.60 increases specificity to 0.8571 while lowering sensitivity to 0.8182. The table demonstrates the trade-off and does not establish a clinical operating point.

8. Automated Checks and Final Result

All data-shape, feature/target, train-test separation, stratification, probability-range, performance, and coefficient-dimension checks passed successfully.

RESULT: The leakage-safe logistic regression model achieved 83.61% test accuracy, 87.88% sensitivity, 78.57% specificity, an F1-score of 0.8529, ROC-AUC of 0.8853, and average precision of 0.9023.

OBSERVATIONS:

- The dataset is only mildly imbalanced, but the dummy classifier shows that accuracy alone can still be misleading.
- Leakage-safe preprocessing is important even though only ten measurement values are missing.
- Logistic regression substantially outperforms the majority-class baseline on every informative test metric.
- At threshold 0.50, sensitivity is higher than specificity and the model produces 4 false negatives and 6 false positives.
- ROC-AUC and average precision show useful ranking performance on the held-out split.
- Cross-validation results are broadly consistent with the held-out results, although uncertainty remains because the dataset is small.
- One-hot encoding prevents integer category codes from being interpreted as equal-interval measurements.
- The fitted coefficients improve transparency but do not establish causal or clinical effects.
- Changing the threshold redistributes false positives and false negatives; no single threshold maximizes every metric.
- External validation, calibration, subgroup evaluation, and clinical review would be necessary before real-world use.

LEARNING OUTCOMES:

- I have learned about logistic regression, the sigmoid function, log-odds, and predicted probabilities.
- I have learned how to build a leakage-safe preprocessing and classification pipeline using Scikit-learn.
- I have learned how to use stratified train-test splitting and compare a trained model with a dummy baseline.
- I have learned how to calculate and interpret accuracy, precision, sensitivity, specificity, F1-score, ROC-AUC, and average precision.
- I have learned how to construct and interpret confusion matrices, ROC curves, and Precision-Recall curves.
- I have learned how to use cross-validation to estimate model stability without including the held-out test set.
- I have learned how to interpret logistic-regression coefficients and odds ratios while considering their limitations.

CONCLUSION:

The completed pipeline demonstrates a reliable logistic-regression workflow for binary classification. Correct feature typing, training-only preprocessing, baseline comparison, multiple evaluation metrics, cross-validation, and explicit threshold analysis make the experiment more trustworthy than reporting accuracy alone. On the fixed test split, the model achieved 83.61% accuracy and an ROC-AUC of 0.8853. The threshold analysis confirms that sensitivity and specificity cannot generally be maximized together. Because the dataset is small and lacks external validation, the result demonstrates machine-learning methodology rather than a deployable medical diagnostic system.